

Thermodynamic State Vector Discrepancy Absolute Difference Math Function in Computational Ecosystems

Lead Scientific Communications & Academic Outreach Agent

Web of Life Research Consortium

<https://github.com/pascalranoroarijaona/WebOfLife>

Sprint 072 Technical Report

Abstract

In computational ecosystems and thermodynamics-based simulation frameworks such as the *Web of Life*, maintaining strict mass conservation and quantifying disequilibrium are foundational for simulating self-organizing biogeochemical cycles (Carbon, Nitrogen, Phosphorus, and Water). Sprint 072 introduces a specialized pure helper function, `computeAbsoluteStockDelta(actual, expected)`, designed to calculate element-wise absolute discrepancies between observed thermodynamic state vectors (`actual`) and baseline or predicted vectors (`expected`). By operating within a pure functional paradigm, this module guarantees deterministic, side-effect-free evaluations that feed regulatory monad feedback loops. These loops drive systemic equilibration toward maximum entropy production limits bounded by external solar forcing, while rigorously enforcing the First and Second Laws of Thermodynamics.

1 Introduction & Systems Ecology Context

Ecosystem stability and thermodynamic efficiency rely heavily on continuous monitoring of matter and energy fluxes. Within the *Web of Life* simulation architecture, metabolic pools, elemental stocks, and trophic tiers are modeled as state vectors subject to rigorous conservation laws.

To evaluate whether a simulated ecosystem transition violates mass-balance constraints or diverges from steady-state expectations, the system requires precise discrepancy metrics. Sprint 072 implements `computeAbsoluteStockDelta`, providing a mathematically robust mechanism to isolate and quantify elemental variances across sparse or complete state vectors.

2 Mathematical Formalization

Let S_A denote the `actual` thermodynamic state vector and S_E denote the `expected` state vector. Each vector maps elemental keys $k \in K$ (e.g., 'C', 'N', 'P', 'H2O') to scalar mass or molar quantities ($\mathbb{R}_{\geq 0}$).

The total universe of keys across both records is defined as:

$$K_{\text{total}} = \text{keys}(S_A) \cup \text{keys}(S_E)$$

For any key $k \in K_{\text{total}}$, if a key is missing from either dictionary, its default quantity evaluates to 0. The absolute stock delta $\Delta(k)$ is formally defined as:

$$\Delta(k) = |S_A(k) - S_E(k)|$$

The resulting delta record Δ_{vector} is constructed as:

$$\Delta_{\text{vector}} = \{k : \Delta(k) \mid \forall k \in K_{\text{total}}\}$$

3 Software Architecture & Implementation

The implementation resides in `src/thermodynamics/state_validator.ts`, adhering to strict functional programming principles and integrating seamlessly with existing monad validation pipelines.

```
/**
 * Computes the absolute stock delta between actual and expected
 * thermodynamic state vectors.
 *
 * @param actual - Record of actual elemental stock quantities.
 * @param expected - Record of expected/baseline elemental stock
 * quantities.
 * @returns A new record mapping each elemental key to its absolute
 * numerical difference.
 */
export function computeAbsoluteStockDelta(
  actual: Record<string, number>,
  expected: Record<string, number>
): Record<string, number> {
  const allKeys = new Set([...Object.keys(actual), ...Object.keys(
    expected)]);
  const deltas: Record<string, number> = {};

  for (const key of allKeys) {
    const actualVal = actual[key] ?? 0;
    const expectedVal = expected[key] ?? 0;
    deltas[key] = Math.abs(actualVal - expectedVal);
  }

  return deltas;
}
```

Listing 1: Implementation of `computeAbsoluteStockDelta` in TypeScript

3.1 Key Architectural Properties

1. **Purity & Immutability:** Inputs (`actual` and `expected`) are never mutated. A fresh record is returned, preserving state transition monad determinism.
2. **Asymmetric Key Resilience:** Utilizing JavaScript `Set` unions combined with nullish coalescing (`?? 0`) ensures sparse state vectors do not trigger runtime reference exceptions.
3. **Floating-Point Precision:** Discrepancies are computed directly in scalar space to facilitate rigorous floating-point tolerance checks ($\epsilon < 10^{-12}$).

4 Thermodynamic & Ecological Implications

- **First Law Compliance (Matter Conservation):** By isolating absolute variances across closed elemental inventories, the engine detects unaccounted sources or sinks of matter, ensuring mass conservation across biogeochemical cycles.
- **Second Law Feedback (Entropy & Dissipation):** The quantified error vector Δ_{vector} serves as a distance-to-equilibrium metric. Regulatory monads ingest these discrepancies to adjust metabolic rates and dissipation pathways, pushing the ecosystem toward steady-state configurations constrained by maximum entropy production (MEP).

5 Conclusion

Sprint 072 establishes a robust, mathematically sound primitive for thermodynamic validation within the *Web of Life* engine. Future sprints will leverage `computeAbsoluteStockDelta` to drive automated adaptive feedback loops, multi-tier trophic balancing, and real-time entropy production monitoring.

Repository Access: For complete source code, test suites, and project documentation, visit the official repository at <https://github.com/pascalranoroarijaona/WebOfLife>.